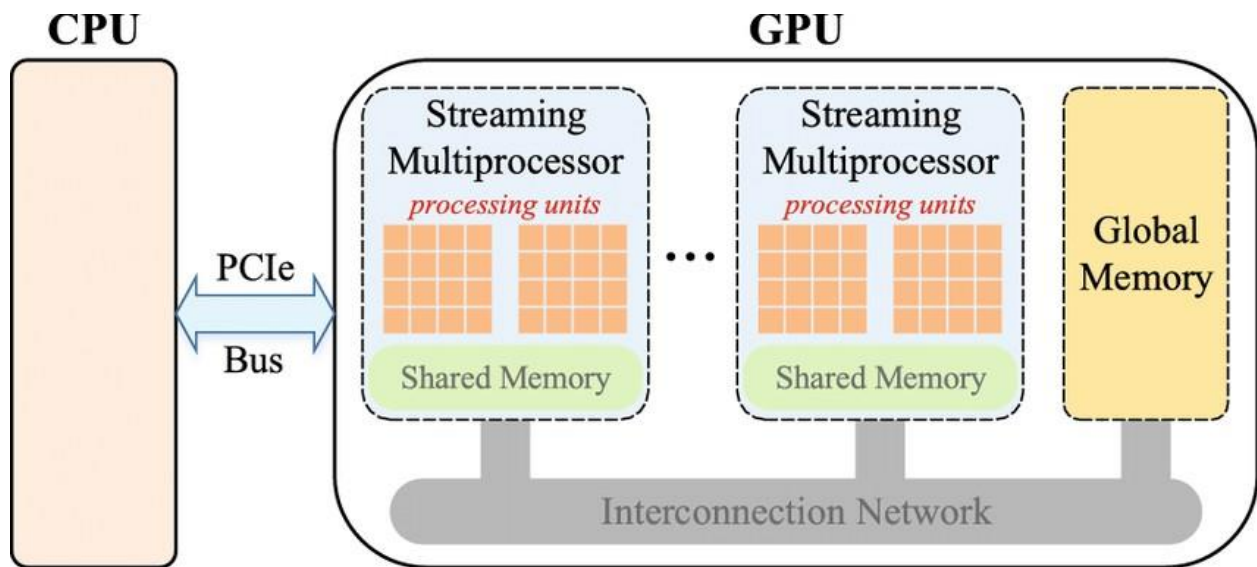


Unit 5: Emerging Processors for AI

1. Deep Dive: AI-oriented Processor Types

We'll explore not just what each processor is, but *how* they are built, what microarchitectural techniques they use, and what trade-offs are involved.

1.1 CPU (Central Processing Unit)



Architecture & Microarchitecture:

- **Control logic, branch prediction, out-of-order execution:**
CPUs are optimized for general-purpose, irregular control flow. They include heavy logic for branch predictors, speculation, multiple issue pipelines, etc.
- **Cache hierarchy:** L1, L2, L3 (sometimes L4). Very fast, small caches close to cores, to reduce latency for frequently used data & instructions.
- **SIMD / Vector Extensions:** AVX, Neon, SVE etc allow CPUs to do some parallel vector computation, but these are limited in width and

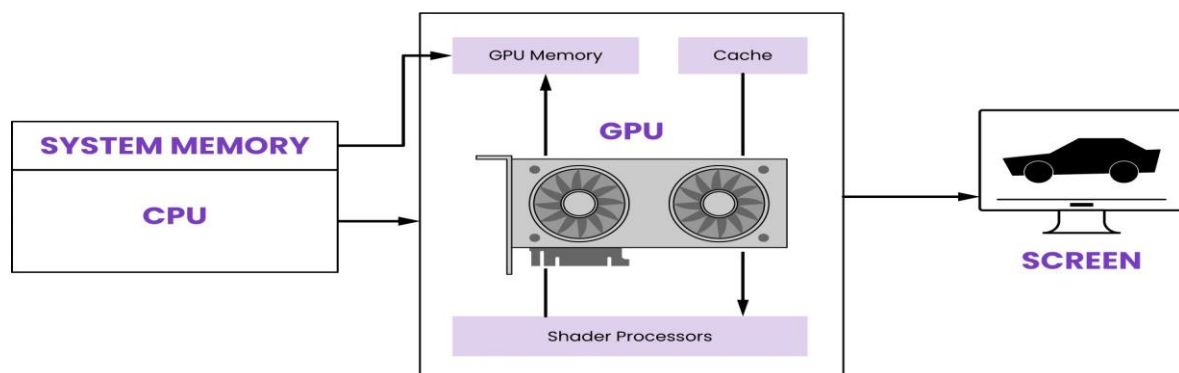
throughput relative to specialized accelerators.

- **High clock speeds**, relatively large per-core performance, but energy/power cost tends to be high per unit of computation.

Strengths & Trade-offs:

- Good at tasks with **branching, control flow, decision logic**, small data, preprocessing, overhead tasks, or orchestrating entire computation pipelines.
- Less efficient when many operations are uniform, regular, massively parallel (e.g., matrix multiply, convolution) — here you get under-utilization of parts of CPU.
- CPUs' power/performance trade-offs: Going to higher clock, wider vectors etc. increases power significantly; thermal design limits.
- Generality vs specialization: CPU supports a wide instruction-set, many languages, and dynamic behavior, but that flexibility comes with hardware cost and inefficiency.

1.2 GPU (Graphics Processing Unit)



Architecture:

- **Many simple cores / ALUs** arranged into groups (e.g., streaming multiprocessors, compute units). Each group handles many threads in parallel.
- **SIMT (Single Instruction, Multiple Threads)** or SIMD style execution: many threads execute same instruction on different data.
- **High bandwidth memory (HBM, GDDR etc.)** to supply data; wide memory buses; specialized memory hierarchies for texture / caching / shared memory inside blocks.
- **Floating point units** optimized for multiple precisions (FP32, FP16, sometimes even lower, sometimes BFLOAT16 or FP8). Also specialized tensor cores in modern GPUs for matrix multiplies.
- Often idle hardware: if kernel has divergent branches, or low arithmetic intensity (operations/data access ratio), then GPU loses efficiency.

Tradeoffs:

- Excellent when tasks are data-parallel, many operations performed on large data sets or large matrices.
- Overheads: loading kernels, memory transfers (host ↔ GPU), communication / synchronization costs.
- Power use is high; cooling, thermal design and power provisioning matter a lot.
- Not as strong in low-latency small or irregular tasks (unless specifically optimized).

1.3 TPU (Tensor Processing Unit)

Architecture & Design Choices:

- ASIC (Application Specific Integrated Circuit) specialized for tensor algebra and deep learning workloads.
- **Systolic arrays or matrix multiply units** with many MAC (multiply-accumulate) units arranged in array, often with specialized data flow to reduce memory fetches.
- Supports lower numeric precision (e.g. bfloat16, mixed precision, INT8 etc.), sometimes sparsity.
- Often large on-chip memory / cache (or weight buffers) to reduce pressure on external memory bandwidth.
- Data flow architectures that allow streaming data in, compute, and stream out results with minimal overhead and latency.

Advantages & Limits:

- Very high throughput for tensor operations; energy efficient for training/inference of deep neural nets.
- Limited flexibility: best suited for workloads that map well to their design (matrix multiplies, CNNs, Transformer etc.). If task is unusual, may perform poorly.
- Often deployed in cloud/data center environments due to size, cost, power.
- Sometimes less mature toolchains; dependence on software frameworks (TensorFlow etc.).

1.4 NPU (Neural Processing Unit)

What makes an NPU special:

- Targeted inference acceleration: optimized for the forward pass of neural networks; sometimes support retraining or small fine-tuning, but usually inference.
- Many cores or MAC units, often arranged as vector or matrix units.

- Use of **reduced numerical precision** (e.g. INT8, INT4, FP16) because inference often tolerates lower precision with acceptable accuracy drop.
- High on-chip memory or scratchpads; specialized data paths; sometimes compression, sparsity support.
- Good power/performance balance for mobile/embedded devices.

Architectural challenges:

- Memory bandwidth and thermal/power constraints are very tight in edge devices.
- Software stack must support quantization, optimizing model graphs, operator fusion, etc.
- Latency overheads from loading models, scheduling tasks.

1.5 VPU (Vision Processing Unit)

Focused on Vision:

- Specialized for operations common in vision: convolution, pooling, feature extraction, image transformations, sometimes also optical flow, motion estimation, etc.
- May have fixed-function units: e.g., image signal processing (ISP) pipelines, dedicated hardware for color conversion, demosaicing, filtering.
- Often optimized for streaming data (camera sensors), real-time processing, low power.

Trade-offs:

- Less flexible outside vision tasks; not always efficient for non-vision AI (unless they support general AI operators).

- Extreme constraints on latency, memory (especially for high resolution images/video), power in mobile/camera/IoT devices.

2. More Detailed Comparative Example Processors / SoCs (Theory + Real)

Let's look at some real or representative examples and what they do under the hood. (Since exact internal architecture details are often proprietary, I'll focus on known public info + plausible generalization.)

Processor/SoC	Composition (CPU, GPU, NPU etc)	Key Architectural Features	Where It Shines	Where It Faces Limitations
Apple M-series (e.g. M3)	Unified SoC with CPU cores (performance + efficiency), GPU, Neural Engine (NPU)	Unified memory architecture (shared memory between GPU/NPU/CPU), deep integration with OS, aggressive use of low-precision and specialized hardware for vision & sensor tasks, tight power & thermal constraints in laptops	Excellent on-device AI, low latency for inference, energy efficient, good for workloads that mix compute + graphics + AI (e.g., ML in photos, offline speech)	Less raw throughput for huge training tasks; floating-point precision may be constrained; expansion limited (not modular)
Intel Core Ultra / Meteor Lake / later	CPU + integrated GPU + built-in AI accelerator (NPU)	Heterogeneous compute, possibly dedicated NPU blocks, shared cache, pipelines to offload AI tasks transparently; software support (drivers,	Better AI features in laptops/desktops; more efficient than just CPU doing everything; helps in tasks like image enhancement, speech, etc.	May lag behind discrete GPUs/TPUs in training performance; thermal constraints; potential software/hardware

		frameworks) important		mismatch overheads
NVIDIA RTX (Discrete GPU + Tensor / RT cores etc.)	Large GPU plus specialized Tensor cores, RT cores, large memory (HBM or GDDR)	High bandwidth, high FP/INT precision, support for mixed precision, supports large batch training, very parallel workloads; mature toolchain (CUDA, cuDNN)	Training large models; inference for large scale; rendering + AI combined; high throughput	Power consumption, heat, cost; not ideal in edge / battery powered devices; latency for small inputs can be less favorable unless pipeline is optimized
Google TPU (v2, v3, v4, Edge TPU etc.)	ASIC built for TensorFlow and tensor workloads; Cloud TPUs vs edge variants	Very large matrix-multiply units; weight reuse; high bandwidth internal memory; optimizations for communication in cloud (interconnects)	Large scale training; inference at scale; efficient for tensor workloads; edge TPUs good for small / real-time inference	Less generality; sometimes limited to particular software stacks; debug / flexibility can be more limited; precision constraints; cost and deployment constraints depend on edge vs cloud version
Qualcomm Hexagon & Similar Mobile AI Engines	DSP + NPU blocks integrated with CPU + GPU + ISP in mobile SoCs	Optimized for power, many low-precision operations, pipelines for real-time vision/voice tasks, use of hardware supported quantization/fused ops	Very efficient inference in mobile devices; low latency; low power; often always-on features; voice wake, etc.	Limited throughput for large model training; memory bandwidth and capacity constraints; sometimes limited software portability; model size constraints; precision vs accuracy trade-offs

3. Edge AI vs. Cloud AI: Deeper Theory and Trade-Offs

Not just simply where computations happen — but the design implications, constraints, and hybrid models.

3.1 Edge AI

Constraints:

1. **Power / Energy Budget**

Many edge devices are battery powered; even if tethered, power may be limited or thermal dissipation is constrained.

2. **Memory Constraints**

RAM is limited. On-chip memory/cache is precious. Model weight size / activations must be small or stored efficiently.

3. **Latency & Determinism**

Real-time constraints: response time must be low and consistent, especially for tasks like autonomous driving, AR, robotics.

4. **Model Size / Complexity**

Must often prune, quantize models; use sparse representations; cascaded models; model compression techniques.

5. **Security & Privacy**

On-device inference avoids transmitting sensitive data; regulatory constraints (e.g. GDPR) may require local processing.

3.2 Cloud AI

Advantages:

1. **Scale & Resources**

Large clusters of GPUs / TPUs; large memory; high bandwidth storage; many servers working in parallel.

2. **Model Training**

Massive datasets; capability to train enormous models with thousands of parameters, possibly across distributed compute.

3. **Software Ecosystem**

Easier to deploy updates; more flexible environment; more storage; better cooling; you can use heavy frameworks.

Challenges:

- Latency from data transmission; network delays; jitter.
- Cost: electricity, cooling, cloud resources.
- Privacy / compliance: moving data.
- Dependence on stable network.

3.3 **Hybrid / Federated / Distributed Models**

- **Distributed inference / training:** Some parts run at edge, some at cloud (e.g., feature extraction on-device, heavy computation in cloud).
- **Model partitioning and offloading:** Splitting neural networks between edge and cloud.
- **Federated learning:** Training happens partly on devices; aggregate updates sent to server; helps with privacy.
- **Caching, incremental inference, batch vs real-time tradeoff.**

4. **Heterogeneous Computing: Theory, Scheduling, Architectures**

Going beyond just having multiple processors, but how to orchestrate, schedule, share resources, manage memory etc.

4.1 **What "Heterogeneous Computing" Means**

- Multiple different types of compute units (CPU, GPU, NPU, perhaps TPU, VPU, DSP) in the same system.
- Each unit is optimized for different kinds of tasks: scalar logic, vector computations, tensor multiplications, specialized vision tasks.
- Unified or discrete memory, interconnects, data movement overheads.

- Software stack and runtime / drivers are critical in deciding how to partition work efficiently.

4.2 Key Architectural Challenges

1. **Memory / Bandwidth Bottlenecks:** Moving data between units is costly (in time, energy). On-chip cache / scratchpads help. Shared memory / zero-copy designs reduce overhead.
2. **Task Scheduling and Partitioning:**
 - Deciding which layer(s) or operations run on CPU vs NPU vs GPU.
 - Data dependencies, pipelining, overlapping compute and data movement.
3. **Precision / Quantization Compatibility:**
 - Different units may support different numeric precision (e.g. CPU maybe FP32/FP64, GPU supports FP16 etc., NPU maybe INT8). Conversion costs, accuracy degradation considerations.
4. **Latency vs Throughput Trade-offs:**
 - Batch processing gives high throughput but increases latency per request.
 - Real-time tasks may require single inference fast rather than batching.
5. **Power & Thermal Management:**
 - Each unit has a power envelope; operating points (frequency, voltage) may be dynamically adjusted.
 - Thermal throttling can reduce performance in mobile/embedded systems.
6. **Software Infrastructure:**
 - Drivers, compiler support, operator libraries, frameworks that can map model graphs onto appropriate units.
 - Toolchain need to support optimizations like kernel fusion, operator scheduling, memory reuse, quantization, pruning.

5. Benchmarks & Metrics: Formal Definitions, Theory, and Use in Practice

Here, more precise definitions, formulas, trade-off curves.

5.1 FLOPS, TOPS, etc.

- **FLOPS:** Floating-Point Operations Per Second. Often used for FP32 (single precision), FP64 (double precision), etc. Measures how many floating-point arithmetic ops a processor can perform per second.
- **TOPS:** Tera (10^{12}) Operations Per Second. Often used for integer or mixed precision (e.g. INT8, INT4) operations in inference settings.
- **Peak vs Sustained:** Manufacturers typically quote *peak* FLOPS/TOPS which assume ideal conditions (all units busy, perfect pipelines, no memory stalls). Real workloads often hit much less.

5.2 Latency, Throughput, Tail Latency

- **Latency:** Time taken for a single inference or task from input to output. Measured e.g. in milliseconds or microseconds.
- **Throughput:** Number of inferences / tasks per unit time (e.g., per second). A GPU or TPU may achieve very high throughput if batched.
- **Tail Latency:** Statistical measure (e.g., 95th percentile, 99th percentile) of latency over many requests. In interactive systems, worst-case latency (tail) is important.
- **Latency vs Throughput trade-off:** You can often trade off throughput for lower latency by reducing batch size etc.

5.3 Power, Energy, Efficiency Metrics

- **Power:** Instantaneous energy usage (Watts). For computing systems includes dynamic and static power.

- **Energy per inference / energy per operation:** How much energy (Joules) is consumed to perform one inference, or one MAC, etc.
- **Performance per watt:** throughput (or operations) per watt. Common metric for comparing efficiency:

$$\text{Perf_per_watt} = \frac{\text{Throughput (e.g. inferences/s) or TOPS}}{\text{Power (Watts)}}$$

- **Thermal Design Power (TDP):** Max power that a chip is designed to dissipate under typical load.
- **Dynamic Voltage and Frequency Scaling (DVFS):** Adjusting voltage/frequency for power/performance trade-offs.

5.4 Other Metrics

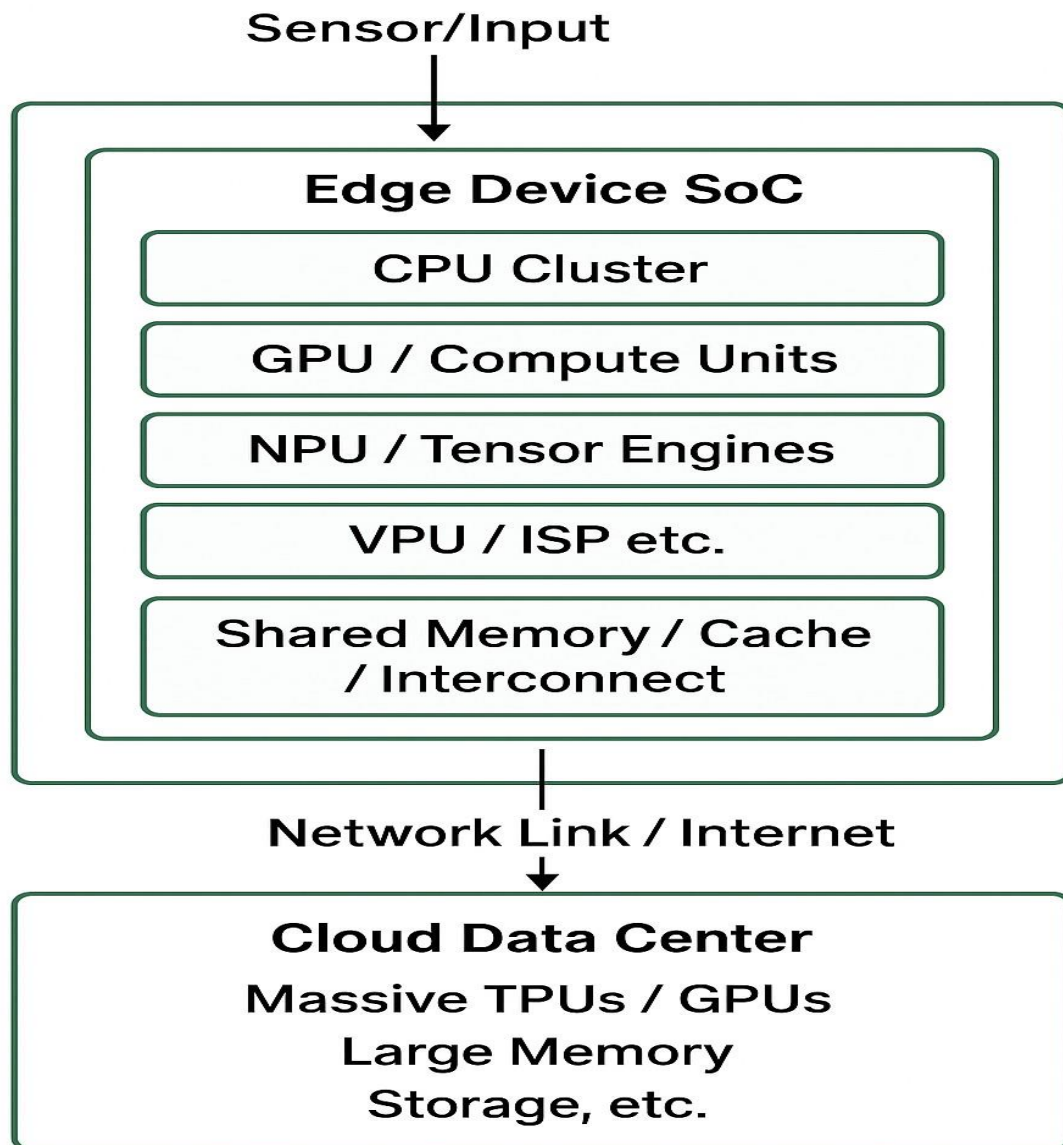
- **Memory footprint:** Size (in MB/GB) of model (weights + activations) during inference or training.
- **Memory bandwidth:** Speed at which data can be moved between memory and compute units.
- **Arithmetic Intensity:** Ratio of computation to memory traffic (e.g. FLOPs per byte transferred). High arithmetic intensity means more computation per data moved — favorable for accelerators.
- **Precision vs Accuracy trade-offs:** Using lower precision (INT8, FP16, 4-bit etc.) reduces compute, memory, power, but might degrade accuracy. Measuring accuracy drop vs savings is important.
- **Scalability:** How performance changes with increasing problem size, batch size, hardware resources.
- **Utilization:** How well the hardware units are kept busy. E.g. if many compute units are idle due to memory stalls or waiting on data, then raw TOPS are not realized.

- **First inference / cold start time:** Load model into memory, initialize accelerators, etc. Especially important in edge or serverless deployment contexts.

6. Putting It All Together: Architecture Diagram (Concept) + Analysis

I'll describe a detailed diagram concept, and then analyze its components deeply.

6.1 Diagram Concept: Heterogeneous AI System Architecture



Some key features in this architecture:

- Shared memory/cache inside the edge SoC: reduces data movement latency between CPU, GPU, NPU.
- Interconnects between compute units: bus or fabric; may have DMA (Direct Memory Access) units to move weights/activations.
- For cloud side: scale via many of such compute units, perhaps in clusters, with high bandwidth internal interconnects.
- Scheduler / operating system: decides where each computation (layer, operator) runs. E.g., some layers on NPU, some pre/post-processing on CPU, etc.

6.2 Analysis of That Diagram: How it works, what bottlenecks/trade-offs are

- **Data Flow & Bandwidth Bottleneck:** If CPU / GPU / NPU share memory but memory bandwidth is limited, then data movement becomes the bottleneck. For example: loading model weights, fetching activations, writing outputs. On-chip caches / scratchpads / weight reuse help.
- **Compute / Memory Balance:** If arithmetic intensity is low (many loads/stores vs compute), then hardware will be underutilized. Good accelerators are optimized for high arithmetic intensity (e.g., large matrices, convolutions etc.).
- **Precision Conversion Overheads:** If CPU handles FP32, but NPU supports INT8, then you have to quantize and dequantize; these conversions cost time/energy and may reduce accuracy or require calibration.
- **Scheduler Overhead:** Dividing work (for example: CPU handles input preprocessing, GPU handles large convolution layers, NPU handles small fully connected / attention layers) introduces overhead: context switching, data movement, synchronization.

- **Latency Sensitivity vs Batch Throughput:** If the system waits to collect a batch to feed to GPU/NPU to improve utilization, latency per request increases. For interactive applications (speech, AR etc.), this latency may be unacceptable.
- **Thermal / Power Envelope Constraints:** Edge devices (mobile, IoT) have limited thermal dissipation, so peak performance cannot be sustained for long. Throttling may reduce performance under load.
- **Accuracy vs Efficiency Trade-Off:** Using lower precision (INT8, 4-bit etc.), pruning, sparsity, approximate computing, reduces power & speed, but may degrade model accuracy. There's often a curve showing diminishing returns.

7. More Benchmark & Metric Theory: Formulas, Empirical Studies

I'll mention some research that shows empirical tradeoffs, and some formal relationships.

7.1 Empirical Benchmarks and Trade-Offs

- Study "*Benchmarking the Performance and Energy Efficiency of AI Accelerators for AI Training*" compares CPUs, GPUs, TPUs across deep learning workloads (CNNs, RNNs, Transformers) and shows that **GPUs/TPUs outperform CPUs** by orders of magnitude in throughput and energy efficiency in large tasks.
- Another study "*Power Estimation and Energy Efficiency of AI Accelerators on Embedded Systems*" looks at edge devices (GPUs, edge TPUs) and different precisions, showing how power consumption scales and which precision levels give best trade-offs.
- "*An Analysis of Deep Neural Network Models for Practical Applications*" shows relationship between inference time, power consumption, model size, number of operations; e.g. energy constraint places upper bound on model complexity.

7.2 Theoretical Relationships and Tradeoffs

- **Arithmetic Intensity Law:** Amdahl's Law like thinking: the benefit of adding more compute power is limited unless you have enough data / arithmetic work to keep the compute units busy. If memory bandwidth or data movement is the bottleneck, more compute units won't help.
- **Roofline Model:** A visual model that plots performance vs arithmetic intensity. There are two regimes:
 1. **Memory-bound regime:** Performance limited by memory bandwidth.
 2. **Compute-bound regime:** Performance limited by compute capacity (FLOPS / TOPS).
- The roofline model helps understand when an algorithm or layer is bounded by memory or compute, helping in hardware/software co-design.
- **Diminishing Returns in Precision Reduction:**

E.g. reducing from FP32 to FP16 gives big energy & speed gains with small accuracy loss; but going further (INT8, INT4, binary etc.) gives decreasing gains and potential large accuracy degradation unless carefully handled.
- **Power vs Frequency / Voltage Curve:** Usually power consumption grows super-linearly with frequency and voltage. So doubling frequency more than doubles power; diminishing returns.
- **Batch Size vs Latency Throughput Tradeoff:** As batch size increases, throughput increases (amortizing overheads) but latency per single sample increases. For real-time, batch size = 1 or small; for throughput, batch big.

8. Additional Concepts

- **Sparsity & Model Compression:** Using weight pruning, structured/unstructured sparsity, sparse matrices, weight sharing, encoding to reduce memory and computation. Highly relevant especially for NPUs and edge devices. These architectures need hardware support for skipping zeros, compressed model storage. (E.g. "EIE: Efficient Inference Engine on Compressed Deep Neural Network" shows large energy savings by compressing networks and doing sparse inference, skipping zero activations.)
- **Mixed Precision & Quantization:** Using different numerical precision for different parts of model; quantized weights, activations; floating-point vs fixed-point arithmetic; scaling; calibration. Critical for inference speed and energy.
- **Tailor-made Operators / Fused Kernels:** Combining multiple operations into one kernel to avoid memory writes between them, to reduce data movement, hence reduce latency and energy.
- **Memory access patterns & data locality:** Architectures optimized to exploit locality are more efficient. E.g. tiling, blocking, reuse of data in registers or small caches.